

NATURAL LANGUAGE PROCESSING · COMPANION READER

Part-of-Speech Tagging

From word classes to HMMs, the Viterbi algorithm, and MEMMs

Session 6 · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. The slides give you the formulas; this reader gives you the *why*. Every slide idea is opened up into plain English, and every slide example is worked out in full, step by step. Colour-coded boxes guide you. The **blue** “intuition” box states the core idea in one breath. The **amber** “everyday picture” box ties it to real life before any math. The **green** “worked example” box solves a problem with real numbers and no skipped steps. The **red** “watch out” box flags the common mistake. The **purple** “key takeaway” box is the one line to remember. Read it next to the slides, in order.

The road map. We start with what a tag is and why tags help. Then we meet word classes and the Penn Treebank tag set. Next we hit the real problem: one word can wear many tags. To solve it we build a Hidden Markov Model, turn it around with Bayes’ rule, and fill its two tables by counting. We work the “race” example, build the tables from a tiny corpus, and play with the ice-cream model. Then we decode with the Viterbi algorithm, in full. We close with the limits of the model and the discriminative answer, the MEMM.

1 What POS tagging is, and why it helps

Every word in a sentence does a job. “Dog” names a thing. “Run” names an action. “The” points at a thing. A **part of speech** (POS) is the name of that job: noun, verb, article, and so on. **POS tagging** is the task of giving each word in a sentence its one correct job label, called a **tag**.

We write the input as a list of words $w_1, w_2, \dots, w_n$. The output is a matching list of tags $t_1, t_2, \dots, t_n$, one tag per word. Here n is the number of words in the sentence. The little numbers are positions, so w_3 is the third word and t_3 is its tag.

The intuition

POS tagging hands every word a name tag that says what role it plays in this sentence. Same words in, same number of tags out, lined up one to one.

★ Everyday picture

Think of a busy kitchen. The same person can be a chef, a waiter, or a cashier, depending on the moment. To run the kitchen you first label who is doing what right now. POS tagging does this for words: it reads the sentence and pins the right role on each word.

Tagging is the first step of many language tasks. It helps cut a word down to its base form (**lemmatization**). It helps a **parser**, the tool that finds sentence structure, because the parser needs to know a noun from a verb first. It helps **Named Entity Recognition** (NER), which finds names of people and places. It also helps machine translation pick the right words.

 Worked example: tag “the lead paint is unsafe”

We tag each word with its job in this sentence.

Step 1. Read the words in order. the, lead, paint, is, unsafe.

Step 2. Label “the”. It points at a thing, so it is a determiner: **the/Det**.

Step 3. Label “lead”. Here it means the metal, a thing, so it is a noun: **lead/N**. (In “lead the team” it would be a verb.)

Step 4. Label “paint”. Also a thing here: **paint/N**.

Step 5. Label “is”. It is the verb: **is/V**.

Step 6. Label “unsafe”. It describes the paint, so it is an adjective: **unsafe/Adj**.

Final tagging: **the/Det lead/N paint/N is/V unsafe/Adj**.

POS tagging: each word gets one tag for this sentence

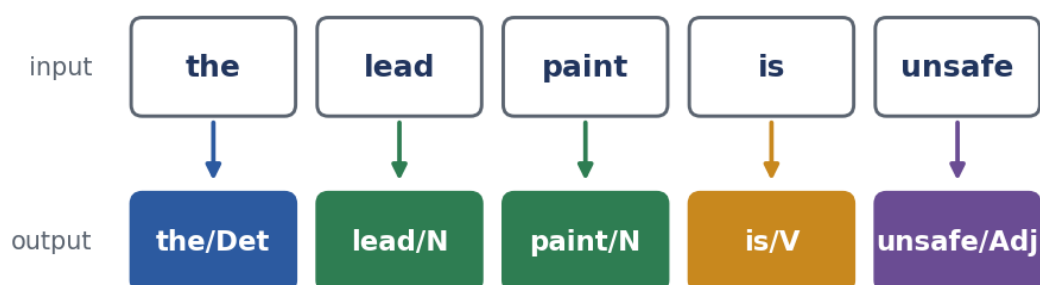


Figure 1: POS tagging takes plain words in and gives back the same words, each carrying one tag. How to read it: the arrow under each word points to that word plus its chosen tag.

 Watch out

A word does not own a fixed tag. “Lead” is a noun in “lead paint” but a verb in “lead the way”. The tag belongs to the word *in this sentence*, not to the word in the dictionary.

Where this shows up in ML. POS tags are an early stage in older language pipelines, feeding parsers, search, and translation. In a modern large language model the tags are not printed. Yet the model still learns these word roles inside its hidden layers. Tagging is also the simplest example of **sequence labeling**, the shape of NER and many other tasks.

 Key takeaway

POS tagging maps each word to one tag for this sentence. It is the first step for parsing, NER, search, and translation.

2 Word classes: open and closed

There are about eight traditional parts of speech: noun, verb, adjective, adverb, preposition, pronoun, article, and conjunction. People also call these word classes or lexical categories. The exact count is debated; we will not join that debate. We only need a clear, fixed set to work with.

The classes split into two groups. A **closed class** has a small, fixed membership. New prepositions almost never appear. An **open class** grows all the time, because people coin new words. English has four open classes: nouns, verbs, adjectives, and adverbs.

The intuition

Closed-class words are the glue of grammar: short, common, and a fixed set, like “of”, “the”, and “can”. Open-class words are the content: the ever-growing pile of nouns and verbs that name new things and acts.

★ Everyday picture

Picture a board game. The rule words on the box never change: “roll”, “pass”, “skip”. That is the closed class. The sticky notes players add for a house rule keep growing forever. That is the open class. Grammar runs on a fixed set of glue words plus an open flood of content words.

Worked example: sort eight words into open or closed

We label each word’s class, then mark it open or closed.

Step 1. “of” — a preposition, a closed class. **Closed.**

Step 2. “the” — an article, a closed class. **Closed.**

Step 3. “she” — a pronoun, a closed class. **Closed.**

Step 4. “can” — an auxiliary verb, a closed class. **Closed.**

Step 5. “laptop” — a noun, an open class. **Open.**

Step 6. “stream” — a verb here, an open class. **Open.**

Step 7. “happy” — an adjective, an open class. **Open.**

Step 8. “quickly” — an adverb, an open class. **Open.**

So four glue words and four content words. New words like “laptop” join open classes; closed classes stay fixed.

Where this shows up in ML. Closed-class words are most of the **stop words** that search engines often drop, since words like “the” carry little topic signal. Word vectors also show the split: closed-class words sit in tight, stable clusters, while open-class words spread out across meaning.

✓ Key takeaway

Closed classes are a small fixed set of glue words; open classes are the growing content words. English has four open classes: noun, verb, adjective, adverb.

3 The Penn Treebank tag set

There are many fine shades of word role, so we must agree on one standard list of tags. A coarse list might use just N, V, Adj, Adv. The most common list in practice is the **Penn Treebank tag set**, which has 45 tags. It splits the coarse classes into finer ones.

For example, the Penn set does not have one “noun” tag. It has NN for a singular noun, NNS

for a plural noun, and NNP for a proper noun like “Paris”. Verbs split the same way. VB is the base form and VBD is past tense. VBG is the “-ing” form and VBZ is third-person singular like “runs”. More tags means more detail for later tools.

The intuition

A tag set is the fixed menu of labels a tagger may choose from. Penn Treebank is a medium menu: detailed enough to be useful, small enough to learn from data.

Penn Treebank: 45 tags — a sample, split open vs closed

OPEN classes (new words appear)

NN dog	NNS dogs	NNP Paris
VB run	VBD ran	VBG running
VBZ runs	JJ happy	RB quickly

CLOSED classes (fixed set)

DT the	IN of	PRP she
MD can	CC and	TO to
CD 42	WP who	

Figure 2: A sample of Penn Treebank tags, split into open classes (left) and closed classes (right). How to read it: each chip shows a tag and one example word; the real set has 45 tags in total.

Worked example: tag a Penn Treebank sentence

We tag “The grand jury commented on a number of other topics .” with Penn tags.

Step 1. **The** — a determiner: **The/DT**.

Step 2. **grand** — an adjective: **grand/JJ**.

Step 3. **jury** — a singular noun: **jury/NN**.

Step 4. **commented** — a past-tense verb: **commented/VBD**.

Step 5. **on** — a preposition, tagged IN: **on/IN**.

Step 6. **a** — a determiner: **a/DT**.

Step 7. **number** — a singular noun: **number/NN**.

Step 8. **of** — a preposition: **of/IN**.

Step 9. **other** — an adjective: **other/JJ**.

Step 10. **topics** — a plural noun: **topics/NNS**.

Step 11. **.** — the full stop has its own tag: **./.**

The full tagging is the/DT grand/JJ jury/NN commented/VBD on/IN a/DT number/NN

of/IN other/JJ topics/NNS ./.

✗ Watch out

One IN tag covers both prepositions and subordinating conjunctions, so “although” in “although I left” is also IN. The word “to” is the exception: it gets its own tag, TO, not IN.

Where this shows up in ML. The tag set is the **label space** of the model: the fixed list of answers it must choose from. A finer set gives downstream tools more to work with but needs more training data per tag. Today many systems use the smaller Universal Dependencies tag set so one model can serve many languages.

✓ Key takeaway

A tag set is the fixed menu of labels. The Penn Treebank set of 45 tags is the common choice. It splits nouns and verbs into fine types.

4 Ambiguity: one word, many tags

Here is the heart of the problem. Many words can take more than one tag. The word “back” is the classic case. It changes its job from sentence to sentence, and only the context tells us which job it has this time.

So the real task is not to look up a word’s tag. The real task is to *choose* the right tag for this use of the word. The fancy name for choosing the right meaning is **disambiguation**, which just means “removing the doubt”.

👉 The intuition

Tagging is choosing, not looking up. A word like “back” brings several possible tags; the sentence around it decides which one fits here.

★ Everyday picture

The word “charge” depends on who says it. A soldier hears “attack”. A shopper hears “pay”. A nurse hears “person in my care”. Same letters, different jobs, and the setting decides. Words in a sentence work the same way.

📝 Worked example: the four tags of “back”

We read each sentence and pick the tag that fits “back” there.

Step 1. “the back door” — “back” describes the door, so it is an adjective: **back/JJ**.

Step 2. “on my back” — “back” is a body part, a thing, so it is a noun: **back/NN**.

Step 3. “win the voters back” — “back” tells how, so it is an adverb: **back/RB**.

Step 4. “promised to back the bill” — “back” is the action, so it is a verb: **back/VB**.

One spelling, four tags. The only thing that changed was the words nearby.

Where this shows up in ML. This is why context models matter. A method that gives each word a single fixed tag will fail on “back”. Word vectors that change with context, like those

One word, four jobs: the tag of “back” depends on context

the back door	JJ	(adjective)
on my back	NN	(noun)
win the voters back	RB	(adverb)
promised to back the bill	VB	(verb)

Figure 3: The word “back” takes four different tags in four sentences. How to read it: each row is a phrase on the left and the tag that “back” earns there on the right.

inside BERT, exist for this reason. The same word should get a different vector, and a different tag, in a different sentence.

✓ Key takeaway

Many words carry several possible tags, so tagging is disambiguation: use the neighbours to choose the right tag for this use.

5 Tagging as a sequence problem

We now see the sentence as a **sequence**: an ordered list of words. We are given the words, and we must pick the best matching list of tags. The two lists have the same length and line up one to one.

Why not tag each word on its own? Because tags lean on their neighbours. After a determiner like “the”, a noun is far more likely than a verb. A good tagger weighs the whole sentence at once, not each word alone.

👉 The intuition

Reading the tags as a sequence lets each choice help the next. The best answer is the whole tag list that fits the whole word list, not five separate guesses.

★ Everyday picture

A crossword puzzle is the same shape. You rarely fill one square from its clue alone. Each letter must agree with the words crossing it. You solve the grid as one joint puzzle. Tagging a sentence is solving such a grid.

The same shape, called **sequence labeling**, shows up far beyond tagging. NER labels each word as part of a person, place, or company name. Semantic role labeling marks who did what to whom. Even genetics uses it, labeling parts of a DNA string. All of these give a label to each item in an ordered list.

A plain question shows why we cannot just try every tag list. Suppose a sentence has n words

and the tag set has T tags. Then the number of possible tag lists is T^n , because each of the n words could take any of the T tags.

Worked example: how many tag lists for a short sentence?

We count the possible tag lists for a 5-word sentence with 45 tags.

Step 1. Set the numbers. Words $n = 5$. Tags $T = 45$.

Step 2. Use the rule. The count is $T^n = 45^5$.

Step 3. Work it out. $45^2 = 2025$, $45^3 = 91,125$, and $45^5 = 45^3 \times 45^2 = 91,125 \times 2025$.

Step 4. Combine. $91,125 \times 2025 \approx 1.85 \times 10^8$.

About 185 million tag lists for a tiny sentence. We need a smart method, not brute force — that is what the rest of this companion builds.

Watch out

Tagging each word with its own most common tag ignores the neighbours. It breaks on words like “back” and “race”, whose tag depends on context. We need a model of the whole sequence.

Where this shows up in ML. Sequence labeling is a core pattern in NLP. NER, chunking, and slot filling for voice assistants all give one label per word. Modern systems use a BiLSTM or a Transformer to read the whole sentence, then label each word. That is just a richer version of the joint idea here.

Key takeaway

Tagging is sequence labeling: choose the best whole tag list for the whole sentence. The count of lists is far too large to test one by one.

6 Markov chains: today depends on yesterday

Before the full tagger, we need one tool: the **Markov chain**. It is a simple model of a process that moves between **states** over time. Its one rule is that the next state depends only on the current state, not on the older past.

We name the pieces. The states are $q_1, q_2, \dots, q_N$, where N is how many states there are. A **transition probability** a_{ij} is the chance of moving from state i to state j . All these numbers form the transition matrix A . A start vector, written π (the Greek letter pi), gives the chance of beginning in each state.

The intuition

A Markov chain has a short memory. To guess tomorrow it looks only at today. The day before is already baked into today, so the chain forgets it.

Everyday picture

Board games like Snakes and Ladders work this way. Your next square depends only on the square you are on now and the dice. How you reached this square does not matter. The chain only cares about where you stand right now.

The “only today matters” rule is the **Markov assumption**. With it, the chance of a whole path of states is just the start probability times one transition for each step. We multiply them because each step is taken as independent given the state before it.

A Markov chain for weather: today depends only on yesterday

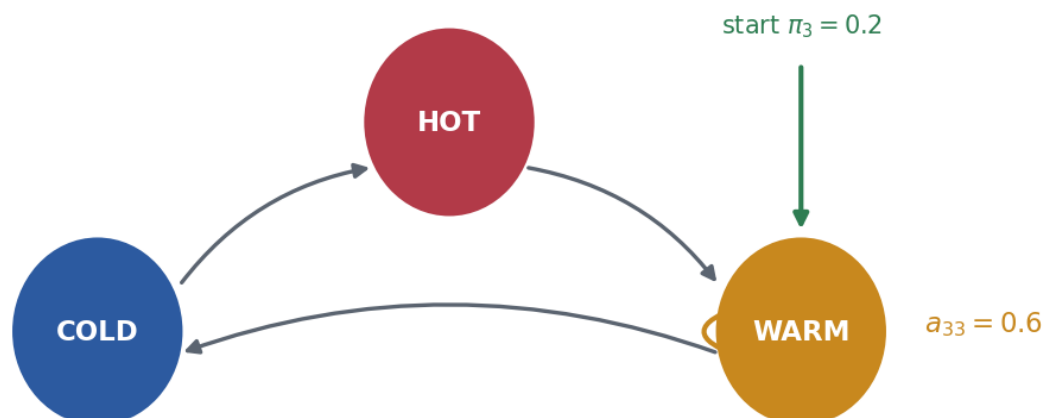


Figure 4: A weather Markov chain with three states. How to read it: arrows are transition chances between states; the green arrow is the start chance for WARM, and the small amber loop is the chance WARM stays WARM.

Worked example: four warm days in a row

We want the chance of warm, warm, warm, warm. Let WARM be state 3, with start $\pi_3 = 0.2$ and stay-warm chance $a_{33} = 0.6$.

Step 1. Write the path. The state sequence is 3, 3, 3, 3.

Step 2. Start in state 3. The first day costs $\pi_3 = 0.2$.

Step 3. Stay warm three times. Each next day costs $a_{33} = 0.6$, and there are three steps: $0.6 \times 0.6 \times 0.6 = 0.6^3$.

Step 4. Multiply it out. $0.6^3 = 0.216$.

Step 5. Combine.

$$P(3, 3, 3, 3) = \pi_3 a_{33}^3 = 0.2 \times 0.216 = \mathbf{0.0432}.$$

So about a 4.3% chance. Each extra warm day multiplies in another 0.6, so long warm spells get unlikely fast.

Where this shows up in ML. Markov chains run under many models. A bigram language model is a Markov chain over words. PageRank is a Markov chain over web pages. Reinforcement learning uses Markov decision processes, which add actions and rewards to this same skeleton.

Key takeaway

A Markov chain moves between states, with one rule: only the current state matters. A path's chance is the start times one transition per step.

7 Hidden Markov Models: the generative story

In a plain Markov chain we can see the states. In tagging we cannot: we see the words, but the tags are hidden. A **Hidden Markov Model** (HMM) is a Markov chain whose states are hidden. It adds a rule for how each hidden state produces a visible output.

So an HMM uses two kinds of information. **Transition** probabilities link a tag to the next tag, written $P(t_i | t_{i-1})$. **Emission** probabilities link a tag to the word it produces, written $P(w_i | t_i)$. The bar “|” means “given”, so $P(w_i | t_i)$ reads “chance of word w_i given tag t_i ”.

The intuition

An HMM tells a little story. A hidden tag picks the next hidden tag, then each tag drops a visible word. We only see the words and must guess the tags that made them.

★ Everyday picture

You are inside a windowless office. You cannot see the sky, but you see coats. On cold days people carry heavy coats; on warm days, none. The weather is hidden; the coats are visible. You read the coats to guess the weather. An HMM reads words to guess tags.

The HMM is a **generative** model: it gives a recipe to build the whole pair of tags and words. The chance of one tag list with one word list multiplies a transition for each tag and an emission for each word:

$$P(w_1 \dots w_n, t_1 \dots t_n) = \prod_{i=1}^n \underbrace{P(t_i | t_{i-1})}_{\text{transition}} \underbrace{P(w_i | t_i)}_{\text{emission}}. \quad (1)$$

The big $\prod$ symbol means “multiply all these together”, once for each position i from 1 to n . We treat the start tag t_0 as a special “ $\langle s \rangle$ ” symbol so t_1 has something before it.

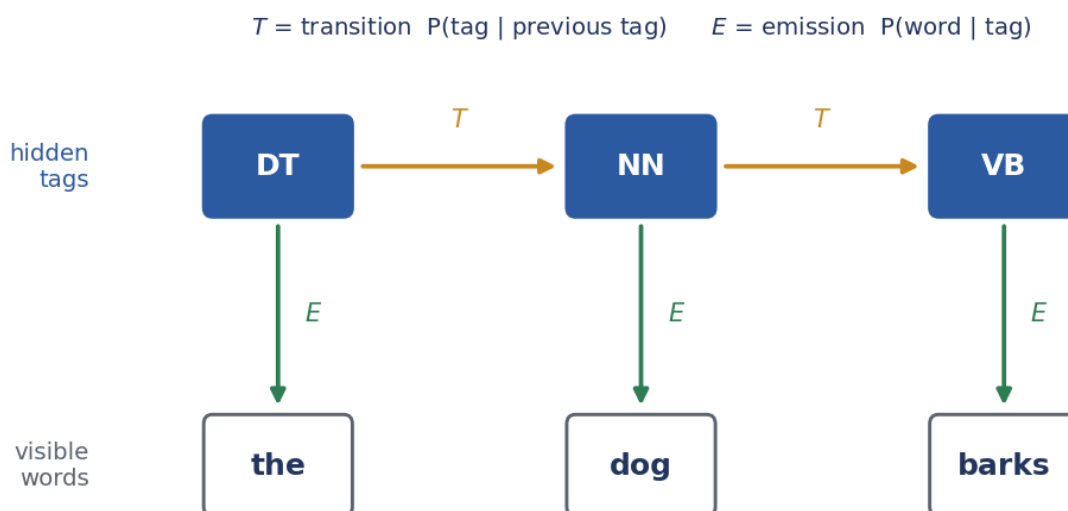


Figure 5: The HMM generative story for “the dog barks”. How to read it: amber arrows across the top are transition chances between hidden tags; green arrows down are emission chances from each tag to its word.

 Worked example: joint chance of “the dog barks” as DT NN VB

Suppose a corpus gives these numbers. Transitions: $P(\text{DT} \mid \langle s \rangle) = 0.3$, $P(\text{NN} \mid \text{DT}) = 0.5$, $P(\text{VB} \mid \text{NN}) = 0.4$. Emissions: $P(\text{the} \mid \text{DT}) = 0.4$, $P(\text{dog} \mid \text{NN}) = 0.02$, $P(\text{barks} \mid \text{VB}) = 0.03$.

Step 1. Multiply the transitions. $0.3 \times 0.5 \times 0.4 = 0.06$.

Step 2. Multiply the emissions. $0.4 \times 0.02 \times 0.03 = 0.00024$.

Step 3. Combine the two parts.

$$P = 0.06 \times 0.00024 = 1.44 \times 10^{-5}.$$

The joint chance is small because we multiply many fractions. We will compare such numbers across tag lists, so the exact size matters less than which list wins.

 Watch out

The HMM rests on two simplifying claims. A tag depends only on the one tag before it. A word depends only on its own tag. Real language bends both rules, but the model still works well and stays easy to compute.

Where this shows up in ML. HMMs were the workhorse of early speech recognition, where the hidden states are sounds and the visible output is audio. They also find genes in DNA. Any time you see a visible stream and want the hidden labels behind it, the HMM is the classic first model.

 Key takeaway

An HMM is a Markov chain with hidden states. It scores a tag-and-word pair by multiplying one transition per tag and one emission per word.

8 Bayes' rule: turning the problem around

We want the most likely tag list given the words. In symbols we want the tag list $\hat{t}_1 \dots \hat{t}_n$ that makes $P(t_1 \dots t_n \mid w_1 \dots w_n)$ as large as possible. The hat “^” marks our best guess. The word **argmax** means “the input that gives the largest value”.

The trouble is that $P(\text{tags} \mid \text{words})$ is hard to get directly. So we flip it with **Bayes' rule**, a basic law of probability that swaps what is given for what is asked:

$$P(t_{1:n} \mid w_{1:n}) = \frac{P(w_{1:n} \mid t_{1:n}) P(t_{1:n})}{P(w_{1:n})}. \quad (2)$$

Here $t_{1:n}$ is short for the whole tag list, and $w_{1:n}$ for the whole word list.

 The intuition

Bayes' rule turns a hard question into two easy ones. How well do these tags explain the words? And how common are these tags to begin with?

★ Everyday picture

A doctor does the same. Given your symptoms, the doctor wants the disease. That is hard to read off directly. So the doctor instead asks: how well does each disease explain these symptoms, and how common is each disease? Then picks the best fit.

📎 Worked example: drop the bottom of Bayes' rule

We simplify the argmax to two terms we can count.

Step 1. Write the goal. $\hat{t}_{1:n} = \operatorname{argmax}_{t_{1:n}} P(t_{1:n} \mid w_{1:n})$.

Step 2. Insert Bayes' rule.

$$\hat{t}_{1:n} = \operatorname{argmax}_{t_{1:n}} \frac{P(w_{1:n} \mid t_{1:n}) P(t_{1:n})}{P(w_{1:n})}.$$

Step 3. Drop the bottom. The words $w_{1:n}$ are fixed, so $P(w_{1:n})$ is the same for every tag list. It cannot change which list wins, so we remove it.

Step 4. Keep two terms.

$$\hat{t}_{1:n} = \operatorname{argmax}_{t_{1:n}} \underbrace{P(w_{1:n} \mid t_{1:n})}_{\text{likelihood}} \underbrace{P(t_{1:n})}_{\text{prior}}.$$

The **likelihood** is how well the tags explain the words. The **prior** is how common the tag list is. These two become the emission and transition tables.

Where this shows up in ML. This is the generative recipe behind naive Bayes and many classifiers. Model the likelihood and the prior, then use Bayes' rule to score each label. Choosing the top scorer this way is called **maximum a posteriori**, or MAP, estimation.

✓ Key takeaway

We want $\operatorname{argmax} P(\text{tags} \mid \text{words})$; Bayes' rule turns it into argmax of likelihood times prior, and the constant bottom term drops out.

9 The two tables, filled by counting

The likelihood and prior become two tables of numbers. The **transition table** holds $P(t_i \mid t_{i-1})$, the chance one tag follows another. The **emission table** holds $P(w_i \mid t_i)$, the chance a tag produces a word. We get both by counting in a corpus, a large body of text already tagged by hand.

To get a transition chance, we count and divide. The chance a noun follows a determiner is

$$P(\text{NN} \mid \text{DT}) = \frac{C(\text{DT}, \text{NN})}{C(\text{DT})}, \quad (3)$$

where $C(\text{DT}, \text{NN})$ counts how often DT is right before NN, and $C(\text{DT})$ counts how often DT appears at all. Emission chances work the same way, counting how often a tag goes with a word.

🧠 The intuition

Each table is just tally marks turned into fractions. Count how often one thing follows another, then divide by how often the first thing happened.

★ Everyday picture

A shop owner learns habits by tallying. Of 100 people who bought bread, 60 also bought milk, so “milk after bread” is 0.6. The transition and emission tables are the same kind of tally, taken over tags and words.

🔗 Worked example: a transition and an emission chance

We read two simple counts off a tagged corpus and turn them into chances.

Step 1. Count for the transition. DT appears 1000 times. It is followed by NN 600 of those times.

Step 2. Divide. $P(\text{NN} \mid \text{DT}) = 600/1000 = \mathbf{0.6}$. So a noun very often follows a determiner.

Step 3. Count for the emission. The tag VBZ (a present-tense verb) appears 500 times. The word is “is” in 300 of them.

Step 4. Divide. $P(\text{is} \mid \text{VBZ}) = 300/500 = \mathbf{0.6}$.

Both tables come from this one move: count the pair, divide by the total. This is called **maximum likelihood estimation**.

⚠ Watch out

If a tag-word pair never appears in the corpus, its count is zero, so its chance is zero. One zero in the product makes the whole tag list score zero. Real taggers fix this with **smoothing**, which shifts a little chance onto unseen pairs.

Where this shows up in ML. Counting then dividing is maximum likelihood estimation, the most basic way to fit a model. The zero-count trap and its smoothing fix appear all over NLP, from n-gram language models to naive Bayes text classifiers.

✓ Key takeaway

Build both tables by counting pairs in a tagged corpus and dividing; guard against zero counts with smoothing.

10 Worked example: disambiguating “race”

Now we use the tables on a real ambiguous word. In “Secretariat is expected to race tomorrow”, is “race” a verb (VB) or a noun (NN)? We decide with the words around it: the “to” before it and the “tomorrow” after it, tagged NR for an adverbial noun.

We compare two small products. For each choice of tag for “race”, we multiply three chances. We take the chance of that tag after “to”, the chance of NR after it, and the chance it emits “race”. The larger product wins.

🔗 The intuition

We are not asking “what is race” in a vacuum. We score the verb story and the noun story against the neighbours, and keep the story with the higher score.

 Worked example: is “race” a verb or a noun?

We use these NLTK numbers: $P(\text{VB} \mid \text{TO}) = 0.83$, $P(\text{NN} \mid \text{TO}) = 0.00047$, $P(\text{NR} \mid \text{VB}) = 0.0027$, $P(\text{NR} \mid \text{NN}) = 0.0012$, $P(\text{race} \mid \text{VB}) = 0.00012$, $P(\text{race} \mid \text{NN}) = 0.00057$.

Step 1. Score the verb path. Multiply the three verb numbers:

$$P(\text{VB} \mid \text{TO}) P(\text{NR} \mid \text{VB}) P(\text{race} \mid \text{VB}) = 0.83 \times 0.0027 \times 0.00012.$$

Step 2. Work it out. $0.83 \times 0.0027 = 0.002241$, then $0.002241 \times 0.00012 \approx 2.7 \times 10^{-7}$.

Step 3. Score the noun path. Multiply the three noun numbers:

$$P(\text{NN} \mid \text{TO}) P(\text{NR} \mid \text{NN}) P(\text{race} \mid \text{NN}) = 0.00047 \times 0.0012 \times 0.00057.$$

Step 4. Work it out. $0.00047 \times 0.0012 = 5.64 \times 10^{-7}$, then $\times 0.00057 \approx 3.2 \times 10^{-10}$.

Step 5. Compare. The verb path (2.7×10^{-7}) is about 850 times the noun path (3.2×10^{-10}).

So we correctly tag **race/VB**. The big winner is $P(\text{VB} \mid \text{TO}) = 0.83$: a verb almost always follows “to”.

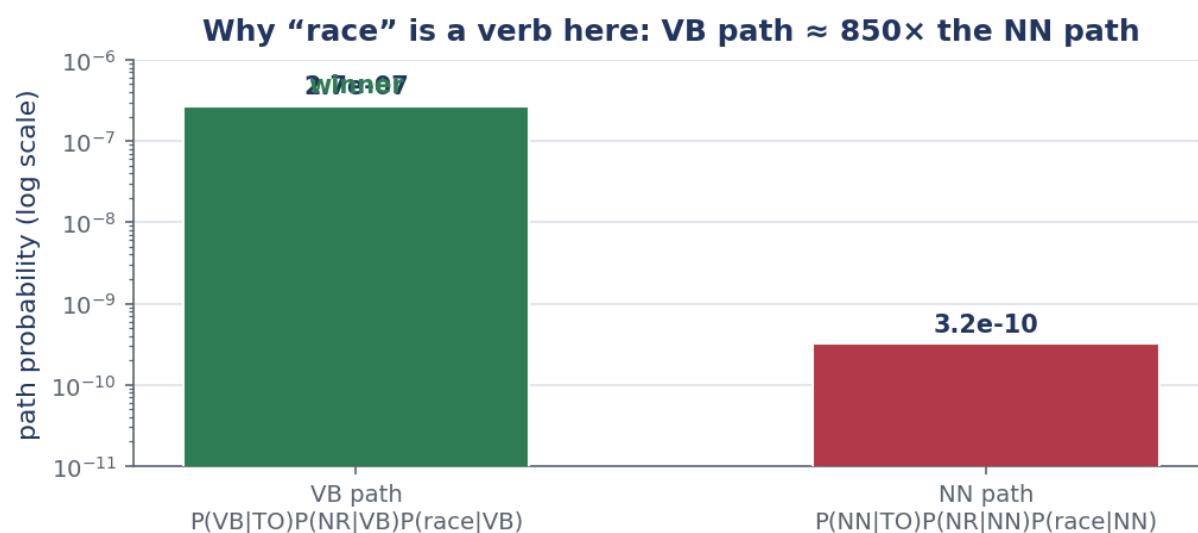


Figure 6: The verb path for “race” beats the noun path by about 850 times. How to read it: each bar is one path’s product on a log scale, so a small bar gap is a large real gap.

Where this shows up in ML. This local scoring is what every tagger and language model does in spirit: read the neighbours, weigh each option, keep the best. Modern models use far richer context, but the move is the same as this three-number comparison.

 Key takeaway

To tag an ambiguous word, score each candidate tag against its neighbours and keep the highest product. Here “to” makes the verb tag win.

11 Worked example: building the tables from a corpus

Let us build the two tables by hand from a tiny corpus. We use three short tagged sentences. Every sentence begins with a start marker “*” and ends with STOP.

- the/DT employees/NNS pass/VB an/DT exam/NN .
- the/DT employees/NNS wait/VB for/IN the/DT pass/VB .
- employers/NNS fire/VB employees/NNS .

To fill the transition table we count tag pairs. The start marker is followed by DT twice and by NNS once, across the three sentences. So three sentence starts give $P(\text{DT} \mid *) = 2/3$ and $P(\text{NNS} \mid *) = 1/3$.

✍ Worked example: count a few table entries

We read counts straight off the three sentences.

Step 1. Start transitions. The marker * starts 3 sentences: 2 go to DT, 1 goes to NNS. So $P(\text{DT} \mid *) = 2/3$ and $P(\text{NNS} \mid *) = 1/3$.

Step 2. After DT. DT appears 4 times. It is followed by NNS twice, by VB once, and by NN once. So $P(\text{NNS} \mid \text{DT}) = 2/4$, $P(\text{VB} \mid \text{DT}) = 1/4$, $P(\text{NN} \mid \text{DT}) = 1/4$.

Step 3. Emission for “the”. DT emits a word 4 times; the word is “the” in 3 of them. So $P(\text{the} \mid \text{DT}) = 3/4$.

Step 4. Emission for “exam”. NN appears once and emits “exam”. So $P(\text{exam} \mid \text{NN}) = 1$.

Reading every pair the same way fills both whole tables, shown next.

Transition counts $P(\text{second tag} \mid \text{first tag})$

	DT	NNS	VB	NN	IN	STOP
*	2/3	1/3				
DT		2/4	1/4	1/4		
NNS			3/4			1/4
VB	1/4	1/4			1/4	1/4
NN						1
IN	1					

Figure 7: The full transition table from the three sentences. How to read it: each row is the first tag, each column the second tag, and the cell is the chance of that pair; darker means a higher chance.

Where this shows up in ML. This by-hand count is exactly what a training script does at scale, over millions of tagged words. The tables here are tiny and full of zeros; a real corpus fills them in and smoothing covers the gaps.

✓ Key takeaway

Build the two tables by tallying tag pairs and tag-word pairs over a tagged corpus. Then divide each count by its row total.

Emission counts $P(\text{word} \mid \text{tag})$

the	3/4					
employees		3/4				
pass			2/4			
an	1/4					
exam				1		
wait			1/4			
for					1	
employers		1/4				
fire			1/4			
.					1	
	DT	NNS	VB	NN	IN	STOP

Figure 8: The full emission table from the three sentences. How to read it: each row is a word, each column a tag, and the cell is the chance the tag emits that word.

12 The ice-cream HMM (Eisner's task)

Here is a friendly way to feel the HMM, due to Jason Eisner. Imagine you are a climate scientist in the far future. You have no weather records for one old summer. But you find a diary that lists how many ice creams someone ate each day. Your job is to guess how hot each day was.

This is an HMM. The hidden states are the weather, HOT or COLD. The visible output is the ice-cream count, 1, 2, or 3. Hot days tend to make more ice creams, so the count leaks information about the hidden weather. Tags and words have this same shape.

The intuition

The ice-cream model is the tagging model in a costume. Hidden weather plays the role of tags; visible ice creams play the role of words. The math does not change.

★ Everyday picture

You guess a friend's mood from how much they text. Chatty often means happy; quiet often means tired. The mood is hidden; the texts are visible. You read the visible clue to guess the hidden cause, exactly the ice-cream setup.

Worked example: score one path: COLD, HOT, COLD for counts 1, 3, 1

We use $P(C \mid \text{start}) = 0.2$, $P(1 \mid C) = 0.5$, $P(H \mid C) = 0.4$, $P(3 \mid H) = 0.4$, $P(C \mid H) = 0.3$.

Step 1. Start COLD. The first state costs $P(C \mid \text{start}) = 0.2$.

Step 2. See 1 on a cold day. Emit: $P(1 \mid C) = 0.5$.

Eisner's ice-cream HMM: hidden weather, visible cones

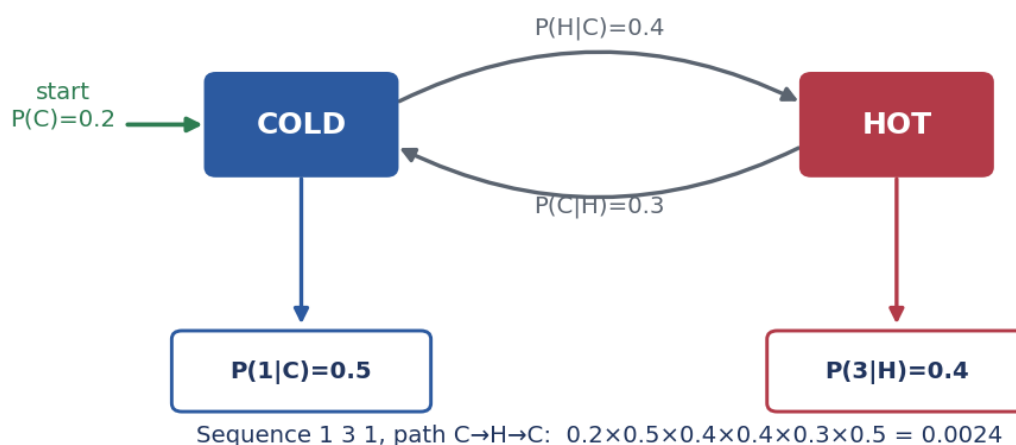


Figure 9: Eisner's ice-cream HMM. How to read it: the two boxes are hidden weather states with transition chances between them; the arrows down are emission chances for the ice-cream counts.

Step 3. Move to HOT. Transition: $P(H | C) = 0.4$.

Step 4. See 3 on a hot day. Emit: $P(3 | H) = 0.4$.

Step 5. Move back to COLD. Transition: $P(C | H) = 0.3$.

Step 6. See 1 on a cold day. Emit: $P(1 | C) = 0.5$.

Step 7. Multiply them all.

$$0.2 \times 0.5 \times 0.4 \times 0.4 \times 0.3 \times 0.5 = \mathbf{0.0024}.$$

That is the score for just one of the eight possible weather paths for “1 3 1”.

⚠ Watch out

For “1 3 1” there are $2^3 = 8$ weather paths. We scored only one. To find the best, scoring all eight works here but blows up for long sequences. That is the push toward the Viterbi algorithm, next.

Where this shows up in ML. The ice-cream task is the standard teaching model for HMM decoding. The same code that guesses weather from ice creams guesses tags from words, or phonemes from audio. Only the tables change.

✓ Key takeaway

The ice-cream HMM is tagging in disguise. Hidden weather is the tags and visible ice creams are the words. One path's score multiplies its transitions and emissions.

13 Why brute force fails: the Viterbi algorithm

Scoring every tag list is too slow. For n words and T tags there are T^n lists. The **Viterbi algorithm** finds the single best list without testing them all. It is a **dynamic programming** method: it solves small overlapping pieces once and reuses the answers.

The key idea is to keep, for each tag at each position, only the best path that ends there. Call it $V[s, t]$: the score of the best path that ends in state (tag) s at time (position) t . We build the next column from the last one:

$$V[s, t] = \max_{s'} V[s', t-1] \cdot P(s | s') \cdot P(w_t | s). \quad (4)$$

Here s' ranges over every tag at the position before. We take the best s' , multiply by the transition into s and the emission of the current word w_t , and store it. We also store which s' won, called a **backpointer**.

The intuition

Many paths flow into a cell, but only the best one can ever be part of the final answer. So we keep just that one per cell, and the work stops exploding.

★ Everyday picture

Plan a cheap road trip across several cities. To reach a city you keep only the cheapest way into it, not every way. You build the cheapest route city by city. Viterbi keeps the best path into each tag, column by column.

Worked example: one Viterbi cell, step by step

Suppose at the word before, two tags have scores $V[\text{Verb}] = 0.03$ and $V[\text{Noun}] = 0.0015$. We want $V[\text{Prep}]$ for the next word “in”. Use $P(\text{Prep} | \text{Verb}) = 0.2$, $P(\text{Prep} | \text{Noun}) = 0.3$, and $P(\text{in} | \text{Prep}) = 1.0$.

Step 1. Path from Verb. $0.03 \times 0.2 = 0.006$.

Step 2. Path from Noun. $0.0015 \times 0.3 = 0.00045$.

Step 3. Take the max. The Verb path (0.006) wins, so the backpointer points to Verb.

Step 4. Times the emission. $V[\text{Prep}] = 0.006 \times 1.0 = \mathbf{0.006}$.

We did one cell with one max, not a search over whole lists. The next section runs this across a full sentence.

Where this shows up in ML. Viterbi decoding is everywhere in sequence models. Conditional random fields use it to find the best label list. Speech systems use it to find the best word path. The same dynamic-programming trick powers CTC decoding in modern speech models.

✓ Key takeaway

Viterbi finds the best tag list in one sweep. It keeps only the best path into each tag, plus a backpointer to rebuild it.

14 Viterbi worked in full: “the doctor is in”

We now decode “the doctor is in” from start to finish. We use the tag set Noun, Verb, Det, Prep, Adv, and a start marker $\langle s \rangle$. We need two tables.

The transition table $P(t_i | t_{i-1})$ gives, among others, $P(\text{Det} | \langle s \rangle) = 0.3$, $P(\text{Noun} | \text{Det}) = 0.9$, $P(\text{Verb} | \text{Det}) = 0.01$, $P(\text{Noun} | \text{Noun}) = 0.2$, $P(\text{Verb} | \text{Noun}) = 0.4$, $P(\text{Prep} | \text{Verb}) = 0.2$, $P(\text{Prep} | \text{Noun}) = 0.3$, $P(\text{Adv} | \text{Verb}) = 0.1$. The emission table $P(w_i | t_i)$ gives $P(\text{the} | \text{Det}) = 0.7$, $P(\text{doctor} | \text{Noun}) = 0.4$, $P(\text{doctor} | \text{Verb}) = 0.1$, $P(\text{is} | \text{Noun}) = 0.1$, $P(\text{is} | \text{Verb}) = 0.9$, $P(\text{in} | \text{Prep}) = 1.0$, $P(\text{in} | \text{Adv}) = 0.1$.

Worked example: fill the Viterbi table for “the doctor is in”

We build one column per word, keeping the best path into each tag.

Step 1. Word 1, “the”. Only Det can emit “the”. $V[\text{Det}] = P(\text{Det} | \langle s \rangle) P(\text{the} | \text{Det}) = 0.3 \times 0.7 = \mathbf{0.21}$. All other tags are 0.

Step 2. Word 2, “doctor”, as Noun. Best comes from Det: $0.21 \times P(\text{Noun} | \text{Det}) \times P(\text{doctor} | \text{Noun}) = 0.21 \times 0.9 \times 0.4 = \mathbf{0.0756}$.

Step 3. Word 2, “doctor”, as Verb. From Det: $0.21 \times 0.01 \times 0.1 = \mathbf{0.00021}$.

Step 4. Word 3, “is”, as Verb. Compare paths from Noun and Verb: from Noun $0.0756 \times 0.4 = 0.03024$; from Verb $0.00021 \times 0.05 \approx 0$. Best is Noun. Times $P(\text{is} | \text{Verb}) = 0.9$: $0.03024 \times 0.9 = \mathbf{0.027216}$.

Step 5. Word 3, “is”, as Noun. From Noun: $0.0756 \times 0.2 \times P(\text{is} | \text{Noun}) = 0.0756 \times 0.2 \times 0.1 = \mathbf{0.001512}$.

Step 6. Word 4, “in”, as Prep. Best from Verb: $0.027216 \times P(\text{Prep} | \text{Verb}) \times P(\text{in} | \text{Prep}) = 0.027216 \times 0.2 \times 1.0 = \mathbf{0.005443}$.

Step 7. Word 4, “in”, as Adv. From Verb: $0.027216 \times 0.1 \times 0.1 = \mathbf{0.000272}$.

Step 8. Pick the winner and backtrace. The largest last-column score is 0.005443 at Prep. Its backpointers run Prep → Verb → Noun → Det.

So the best tagging is **the/Det doctor/Noun is/Verb in/Prep**.

Watch out

Viterbi gives the best whole list, which is not always each word’s own top tag. A tag that looks weak alone can win if it makes the rest of the path much stronger. Always read the path, not single cells.

Where this shows up in ML. This exact table-filling runs inside taggers, chunkers, and named-entity systems built on HMMs or conditional random fields. The numbers come from a trained model, but the sweep and backtrace are the same as here.

Key takeaway

Fill the table left to right with one max per cell. Then backtrace from the best final cell. Here the answer is the/Det doctor/Noun is/Verb in/Prep.

15 Beyond HMMs: bidirectionality and the MEMM

The HMM is a fine first tagger, but it has limits. It struggles with **unknown words** that never appeared in training, since their emission counts are zero. It has **limited context**: a tag sees only the one tag before it. And it suffers from **sparsity**, where many real tag or word pairs are missing from the corpus.

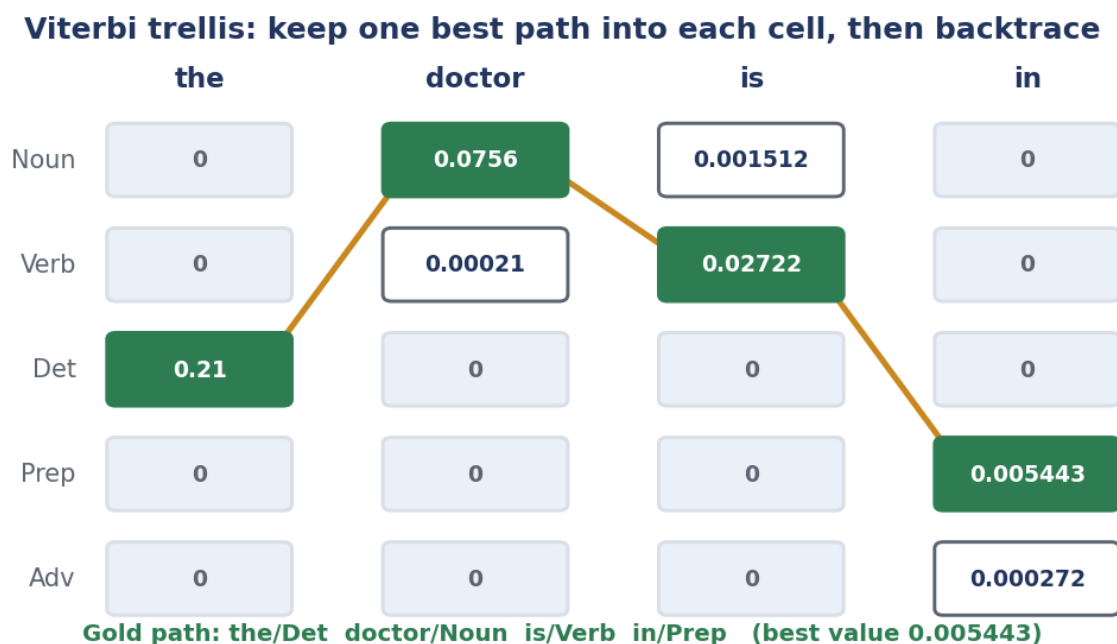


Figure 10: The filled Viterbi table for “the doctor is in”. How to read it: each cell is the best score for that tag at that word; the green cells and amber line are the winning path found by backtracing.

The HMM also runs left to right only. We can add **bidirectionality** by making two passes. One pass reads left to right; another reads right to left. Then we keep the higher-scoring of the two tag lists. Modern taggers usually run both ways.

The intuition

The HMM looks only backward and only at tags. We want a tagger that can look both ways. It should use any clue, like the word’s spelling, not just the tag before it.

A stronger model is the **Maximum Entropy Markov Model** (MEMM). The HMM is generative: it models how words are produced from tags, the likelihood $P(\text{word} \mid \text{tag})$. The MEMM is **discriminative**: it models the answer directly, $P(t_i \mid w_i, t_{i-1})$, the chance of a tag given the word and the tag before. It is logistic regression run along the sentence.

The MEMM’s strength is **features**. A feature is any clue you choose, turned into a yes/no test. For the word “back” it might use: the previous tag is MD, the word ends in “-ck”, the word is lowercase. Each feature gets a learned weight θ (the Greek letter theta), and the weights are trained by gradient descent.

Worked example: score a word with two features

Suppose for one word the model weighs tag VB with score 2.0 and tag NN with score 0.5, from its features. We turn scores into chances with softmax.

Step 1. Raise e to each score. $e^{2.0} \approx 7.39$ and $e^{0.5} \approx 1.65$.

Step 2. Add them. The total is $7.39 + 1.65 = 9.04$.

Step 3. Divide to get chances. $P(\text{VB}) = 7.39/9.04 \approx 0.82$ and $P(\text{NN}) = 1.65/9.04 \approx 0.18$.

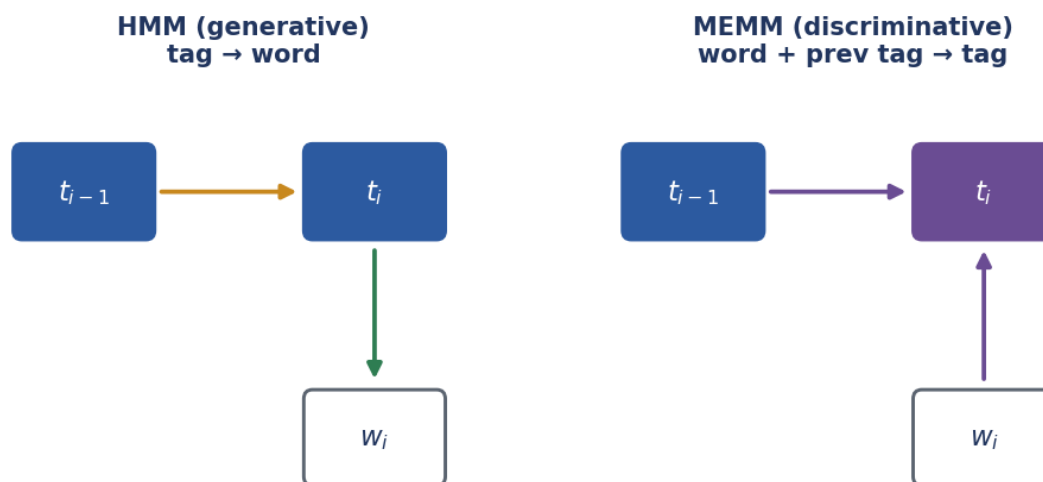


Figure 11: HMM versus MEMM. How to read it: in the HMM the arrow goes from tag down to word (it generates the word); in the MEMM the arrows go from the word and the previous tag up into the tag (it predicts the tag).

Step 4. Check. The two chances add to 1.00, as they must.

The model picks VB with chance 0.82. Richer features give sharper, better splits.

To decode an MEMM we again want the best whole list, so we again use Viterbi. The cheap shortcut is **greedy decoding**: tag the first word, lock it, then the next, and so on. It is fast but cannot undo an early mistake, so it scores worse. Viterbi over the MEMM scores keeps the best whole-sentence list, just like with the HMM.

⚠ Watch out

Greedy decoding makes a hard choice on each word before seeing the rest, so one early error spreads. Use Viterbi to weigh the whole sentence. The MEMM also has a known flaw, the label-bias problem, which later models like the CRF fix.

Where this shows up in ML. The MEMM is logistic regression turned into a sequence model. It leads straight to the conditional random field, and then to neural taggers. The feature idea lives on: today a neural network learns the features itself instead of you writing them by hand.

✓ Key takeaway

HMMs are generative and use Viterbi; MEMMs are discriminative, add free-form features, and also decode with Viterbi rather than greedily.